Project 1: Business FAQ & Lead-Capture Chatbot

Full Architecture + Build Roadmap

Positioning: "AI Customer Support & Lead Capture Chatbot for Small Businesses" — pick one niche business type for the demo (e.g. a restaurant, a clinic, or a real estate agency) so it feels like a real product, not a generic bot.

1. Architecture Overview

Tech stack

Backend: Python + FastAPI (you already know this from your existing chatbot project)
Frontend: Simple HTML/CSS/JS chat widget (embeddable — this is what makes it "sellable," client can drop it into their own site)
Data validation: Pydantic models
Storage: SQLite for MVP (leads + conversation logs), upgrade to PostgreSQL later if needed
NLP approach: Start rule-based/intent-matching (fast, no API cost, reliable for demo) → optionally add an LLM API fallback for open-ended questions (v2)
Deployment: Backend on Render/Railway, frontend on Vercel/Netlify (or serve both from one Render service to keep it simple)

2. Core Features (MVP scope — keep this tight)

Greet visitor + present quick-reply options (Menu/Booking/Hours/Contact — depends on business type)
Answer FAQs from a predefined knowledge base (JSON/DB-backed, easy for client to edit later)
Detect "lead" intent (user wants a callback/booking) → capture name, phone/email → store in DB
Fallback message when it doesn't understand ("Let me connect you to a human" + capture contact)

Simple admin view (even just a `/leads` endpoint or a tiny table page) to show captured leads — this is a strong selling point for clients

3. Step-by-Step Build Roadmap (5 days)

Day 1: Backend Foundation

- [ ] Set up FastAPI project structure (`main.py`, `models.py`, `routes/`, `data/`)
- [ ] Define Pydantic models: `ChatMessage`, `ChatResponse`, `LeadCapture`
- [ ] Build FAQ knowledge base as a JSON file (10-15 Q&A pairs for your chosen niche, e.g. restaurant hours/menu/reservations)
- [ ] Implement basic intent matching (keyword/fuzzy matching — use `rapidfuzz` or simple keyword scoring; no need for heavy ML here)
- [ ] Build `/chat` POST endpoint: takes message + session_id → returns response

Day 2: Lead Capture + Storage

- [ ] Set up SQLite with SQLAlchemy (tables: `conversations`, `leads`)
- [ ] Add conversation state tracking (session_id based) so the bot remembers context within a chat
- [ ] Implement lead-capture flow: detect intent → ask for name/contact → validate input (Pydantic) → save to DB
- [ ] Add `/leads` endpoint (simple GET, maybe with basic auth) to view captured leads
- [ ] Write basic tests for the endpoints

Day 3: Frontend Chat Widget

- [ ] Build a clean, embeddable chat widget (floating button → expands to chat window)
- [ ] Vanilla JS + fetch API to call your `/chat` endpoint (keeps it framework-free and easy to embed anywhere)
- [ ] Style it professionally — this is what the client sees first, so polish matters more than backend elegance here
- [ ] Add typing indicator, smooth animations, mobile-responsive layout

Day 4: Polish + Optional LLM Layer

- [ ] Connect frontend + backend fully, test end-to-end conversation flows
- [ ] (Optional, adds "AI" credibility) Add an LLM API fallback (e.g., Groq's free tier or OpenAI) for questions the rule-based system doesn't catch — keep it scoped so costs stay near zero
- [ ] Add error handling (API down, invalid input, empty messages)
- [ ] Write a clean `README.md` with screenshots + setup instructions

Day 5: Deployment + Demo Assets

- [ ] Deploy backend to Render (free tier) — set environment variables, test live endpoint
- [ ] Deploy frontend to Vercel/Netlify, point it at the live backend URL
- [ ] Test the full live flow end-to-end (this catches CORS issues — set them up properly in FastAPI)
- [ ] Record a 30-60 second demo video (Loom): show a visitor asking FAQs, then triggering lead capture
- [ ] Write the "business problem → solution → result" pitch for your portfolio (e.g., "Reduces repetitive customer queries by X%, captures leads 24/7 even outside business hours")

4. Deployment Checklist (Day 5 detail)

Backend (Render)

Push code to GitHub
Create new Web Service on Render → connect repo
Set build command: `pip install -r requirements.txt`
Set start command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
Add environment variables (LLM API key if used)
Note the live URL (e.g., `https://your-chatbot.onrender.com`)
Frontend (Vercel/Netlify)
Update the widget's JS to point `fetch()` calls to your Render backend URL
Enable CORS on FastAPI backend for your frontend domain
Deploy static files (drag-and-drop on Netlify, or connect GitHub repo on Vercel)
Free tier note: Render's free tier sleeps after inactivity — for demo purposes, ping it before showing a client, or mention this to the client as an upgrade path (paid tier = always-on).

5. What Makes This "Sellable" (keep in mind while building)

Embeddable widget — client should be able to add one `<script>` tag to their existing website. This is the difference between a "cool project" and a "product I can sell."
Editable knowledge base — structure the FAQ data so a non-technical client could theoretically update it (or you offer that as a paid add-on)
Lead capture is the real value — most clients care less about "AI" and more about "does this get me more bookings/inquiries." Frame your pitch around that.

Next possible steps

FAQ knowledge base content for a specific niche (restaurant/clinic/real estate)
The actual FastAPI code for the `/chat` endpoint
CORS + deployment config files